

Assignment #1

ECE 422C (Prof. Edison Thomaz) - The University of Texas at Austin - Fall 2026

Due date: Sept 8th 2026

Checkout: the week after the assignment is due, you will demo your tests to a TA and walk through the errors you found in the starter code

Problem 1 (35 points) - Testing Adjacent-Digit Products

Consider the following input:

```
73167176531330624919225119674426574742355349194934
96983520312774506326239578318016984801869478851843
85861560789112949495459501737958331952853208805511
12540698747158523863050715693290963295227443043557
66896648950445244523161731856403098711121722383113
62229893423380308135336276614282806444486645238749
30358907296290491560440772390713810515859307960866
70172427121883998797908792274921901699720888093776
65727333001053367881220235421809751254540594752243
52584907711670556013604839586446706324415722155397
53697817977846174064955149290862569321978468622482
83972241375657056057490261407972968652414535100474
82166370484403199890008895243450658541227588666881
16427171479924442928230863465674813919123162824586
17866458359124566529476545682848912883142607690042
24219022671055626321111109370544217506941658960408
07198403850962455444362981230987879927244284909188
84580156166097919133875499200524063689912560717606
05886116467109405077541002256983155200055935729725
71636269561882670428252483600823257530420752963450
```

The 4 adjacent digits in the 1000-digit number that have the greatest product are $9 \times 9 \times 8 \times 9 = 5832$.

The required behavior is to find the greatest product formed by at most n adjacent digits in the input string, where n is between 2 and 20. For example, a shorter sequence may be preferable when a longer sequence includes a zero.

Your Task

- You are provided with Problem1.java. The implementation compiles and produces reasonable-looking results, but it does not fully satisfy the specification.
- Write a Java test program named Problem1Tester.java that thoroughly tests Problem1.largestProduct(String s, int n). You do not need JUnit or any other testing framework. A small TestSupport.java helper is provided; you may use it or write equivalent testing code of your own.
- Your tests should include the provided example, small inputs for which the correct answer can be determined by inspection, boundary cases, cases involving zeros, cases in which the best answer uses fewer than n digits, and any other cases needed to distinguish a correct implementation from plausible incorrect ones.
- Your test program must pass when run against a correct implementation and must fail against the provided starter implementation. During grading, your tests may also be run against several hidden faulty implementations. Credit for the testing portion will depend on how many distinct specification violations your tests detect, not simply on the number of test cases you write.
- After your tests are complete, repair Problem1.java so that all of your tests pass. Do not weaken or remove a correct test merely to make your implementation pass.

Inputs: n (int), $2 \leq n \leq 20$; s (String), a 1000-digit number for the official input. Console input format remains n on one line followed by s on the next line.

Output: Print the largest product formed by at most n adjacent digits on its own line.

Problem 2 (35 points) - Testing \$1.00 Words

A \$1.00 word is a word whose letters sum to exactly 100 cents. Each letter is worth its position in the alphabet: $a=1$, $b=2$, ..., $z=26$. Uppercase and lowercase letters have the same value. All non-alphabetic characters have value 0. Words are delimited using spaces.

Example input: The wicked wizard's wily wraith garnishes his master's pasta with garlic.

Example output:

wizard's
garnishes

Your Task

- You are provided with Problem2.java. The implementation is intentionally imperfect.
- Write a Java test program named Problem2Tester.java that thoroughly tests Problem2.findDollarWords(String s). You do not need JUnit or any other testing framework. A small TestSupport.java helper is provided; you may use it or write equivalent testing code of your own.
- Your tests should cover uppercase/lowercase behavior, punctuation and other non-alphabetic characters, multiple matching words, preservation of each original word exactly as it appeared in the input, empty input, spacing-related edge cases consistent with the specification, and other cases needed to distinguish correct behavior from plausible mistakes.
- Your test program must pass against a correct implementation and fail against the provided starter implementation. During grading, your tests may also be run against hidden faulty implementations. Strong suites detect multiple independent defects using small, targeted cases whose expected outputs are easy to justify.
- After your tests are complete, repair Problem2.java so that all of your tests pass. Do not weaken a correct test merely to make your implementation pass. Preserve the required console behavior in main().

Input: s (String), an English paragraph entered through standard input. Constraint: $0 \leq s.length() \leq 10^6$.

Output: Print each \$1.00 word exactly as it appears in the input, one per line.

Problem 3 (30 points) - Part-of-Speech Tagging

Write a Java program that takes as input a sentence as a String, parses each word in the sentence, and identifies what part of speech each word is. You will need to research part-of-speech tagging, choose and download a Java-based part-of-speech tagger, and incorporate it into your program. For example, you could use the Stanford Log-linear Part-Of-Speech Tagger or another suitable Java library. Submit an executable JAR file that includes the tagger you used in addition to the source code.

Example input: I was slowly walking to the park with my over enthusiastic dog when he bit me, and I shouted, Ouch!

Example output: I_PRP was_VBD slowly_RB walking_VBG to_TO the_DT park_NN with_IN my_PRP\$ over_IN enthusiastic_JJ dog_NN when_WRB he_PRP bit_VBD me_PRP ,_, and_CC I_PRP shouted_VBD ,_, Ouch_NNP !_.

Input: s (String), entered via standard input.

Output: A printed and tagged line to the console. Exact tags may vary depending on the library/model used.

Clarifications for Problems 1-3

1. Arguments should be read through the console as described in each problem. In main(), call another method that performs the core work.
2. You may assume that all inputs will be entered correctly.
3. For Problems 1 and 2, the core methods in the starter code are intentionally separated from main() so that they can be tested directly from another Java class.
4. No external testing framework is required. You may use the provided TestSupport.java helper or equivalent ordinary Java code. Your tester must terminate with a nonzero exit status when any check fails so that it can be evaluated automatically.

Instructions for writing code and submitting your files

1. Use package name assignment1. Top-level files containing main() must be named ProblemX.java, where X is the problem number.
2. Submit a ZIP file named Project1_UTEID.zip on Gradescope and Canvas. Problems 1 and 2 may be configured as separate Gradescope submissions; follow the posted submission links.
3. Include Problem1Tester.java and Problem2Tester.java. Your test programs are part of the graded work, not merely a development aid. Include TestSupport.java if your testers depend on it.
4. Include a plain-text defect log named defects.txt. For each error you found in the starter implementations of Problems 1 and 2, give one or two lines stating what the specification requires, what the starter code did instead, and which of your tests exposes it. You will walk through this log during your checkout.
5. Include this line in the header comments of your code: /* Student Name: <your name>, Lab Section: <your lab section #> */.
6. Your code will be compiled and run on the official lab configuration. Make sure both your implementation and tests compile there.

Submission Format

```
Project1_UTEID.zip
assignment1/
  Problem1.java
  Problem1Tester.java
  Problem2.java
  Problem2Tester.java
  TestSupport.java (if used)
  defects.txt
```

Problem3.java
Problem3.jar

How the testing portion will be graded

- Your tests will be evaluated for fault-detection power, not for raw test count.
- In addition to the starter implementation, the grading system may substitute hidden implementations containing different mistakes. Your tester should fail when an implementation violates the specification and pass when it satisfies the specification.
- Hidden faulty implementations may contain off-by-one errors, incorrect handling of zeros or punctuation, incorrect interpretation of 'at most n', output-formatting mistakes, case-conversion mistakes, or other plausible defects. This list is illustrative, not exhaustive.
- A test that depends on private fields, source-code structure, or a particular algorithm is weaker than a test based only on specified behavior and may not work against substituted implementations.
- You may be asked during checkout to explain two or three of your tests: what requirement each test represents, what kind of defect it is intended to expose, and why the expected result is correct.